The Von Neumann Architecture

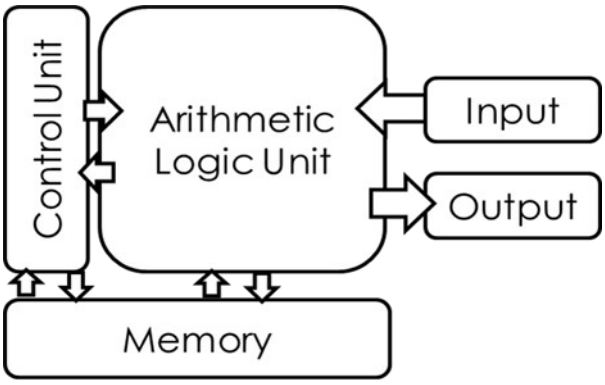
Von Neumann

Before 1945, computers were essentially preprogramed machines. This meant that a computer would typically be set up, with wires and switches to perform a specific task. In this way, a computer program was considered to be part of the machine. The only thing it would be given is data to be processed.

In 1945, a mathematician from the USA called John Von Neumann, had an idea. He wondered if it would be possible to create a computer where the program (and its data) could be stored together, independent of 'the machine'. This meant that the same computer could work, no matter what program it was given. No more hours setting up machines, instead, the time would be spent on creating the program instructions!

Von Neumann Architecture

In the Von Neumann architecture, the data and instructions could be held together in memory and then fetched, decoded and executed one at a time. This meant that simple programs could be written containing both instructions and data and be executed by a computer without having to rewire the circuitry of the machine. It wasn't long before computer scientists invented a simple coding language which would make programming software, accessible to the masses.



Assembly Language and the Von Neumann Architecture

All 'Von Neumann' CPUs have an instruction set. This means that it has a list of instructions that it understands and knows what to do with. Surprisingly, a CPU's instruction set only consists of a few simple instructions.

Here is an example instruction set:

Instruction	Purpose
ADD	Add numbers together
SUB	Subtract numbers from each other
STA	Store value in the accumulator
LDA	Load value from the accumulator
BRA	Branch to a different part of the program (often used to repeat code)
OUT	Output a value (after processing)

A CPU is able to perform each of these instructions and it does so using some very important parts of the Von Neumann Architecture. These parts are called registers, which are simply tiny memory locations (memory stores). The important ones have special names. The following registers are used during each Fetch-Decode-Execute cycle in order to carry out each instruction:

Computer Science UK Membership Site Licence: Do not share outside your centre.

The Accumulator (A)

The Program Counter (C)

The Memory Address Register (MAR)

The Instruction Register (IR)

The Memory Data Register (MDR)

Let's take a look at how these registers carry out simple instructions in a typical F-D-E cycle.

An Example of the Von Neumann Architecture at work...

Below is an example of some instructions and data from a program which has just been loaded into the CPU ready for processing.

The loaded program in CPU Memory:
Memory locations 00-02 currently hold instructions and locations 04-05 hold the data that the instructions will work on.

Address	00	01	02	03	04
Instruction / Data	LDA 4	SUB 5	STA 6		5
Address	05	06	07	08	09
Instruction / Data	2				

Specialised CPU Registers:
Before the CPU fetches the first instruction, these specialised registers (as discussed on the last slide) start off empty.

The Program Counter (C)	
The Memory Address Register (MAR)	
The Memory Data Register (MDR)	
The Instruction Register (IR)	
The Accumulator (A)	

FETCH STAGE

In the first part of the cycle, the instructions and data are fetched from the CPU memory and loaded into the specialised registers:

The program counter starts at 0 and its purpose is to keep track of the number of steps carried out.

The Program Counter (C)	0
The Memory Address Register (MAR)	
The Memory Data Register (MDR)	
The Instruction Register (IR)	
The Accumulator (A)	

Address	00	01	02	03	04
Instruction / Data	LDA 4	SUB 5	STA 6		5
Address	05	06	07	08	09
Instruction / Data	2				

As it's the first step, the counter value is copied to the MAR. This represents the address of the first instruction to be fetched.

The Program Counter (C)	0
The Memory Address Register (MAR)	0
The Memory Data Register (MDR)	
The Instruction Register (IR)	
The Accumulator (A)	

Address	00	01	02	03	04
Instruction / Data	LDA 4	SUB 5	STA 6		5
Address	05	06	07	08	09
Instruction / Data	2				

The program uses the value in the MAR to look up the appropriate instruction by looking in address 00.

The Program Counter (C)	0
The Memory Address Register (MAR)	0
The Memory Data Register (MDR)	
The Instruction Register (IR)	
The Accumulator (A)	

Address	00	01	02	03	04
Instruction / Data	LDA 4	SUB 5	STA 6		5
Address	05	06	07	08	09
Instruction / Data	2				

This instruction is then loaded into the MDR...

The Program Counter (C)	0
The Memory Address Register (MAR)	0
The Memory Data Register (MDR)	LDA 4
The Instruction Register (IR)	LDA 4
The Accumulator (A)	

Address	00	01	02	03	04
Instruction / Data	LDA 4	SUB 5	STA 6		5
Address	05	06	07	08	09
Instruction / Data	2				

...and then moved to the IR, ready to be decoded by the CPU.

DECODE STAGE

Now that the CPU has fetched the first instruction, it needs to decode it.

The Program Counter (C)	0
The Memory Address Register (MAR)	0
The Memory Data Register (MDR)	LDA 4
The Instruction Register (IR)	LDA 4
The Accumulator (A)	

It does this by looking up the IR instruction in its character set. It recognises LDA 4 as meaning LOAD the contents of address 04.

The Instruction Set

Instruction	Purpose
ADD	Add numbers together
SUB	Subtract numbers from each other
STA	Store a value
LDA	Load a value
BRA	Branch to a different part of the program (often used to repeat code)
OUT	Output a value (after processing)

The CPU has now decoded the first instruction and is ready to execute it!

EXECUTE STAGE

Now that the CPU has made sense of the instruction, it now needs to execute the instruction.

The instruction is execute and the value of address 04 is LOADED into the accumulator.

The Program Counter (C)	0
The Memory Address Register (MAR)	0
The Memory Data Register (MDR)	LDA 4
The Instruction Register (IR)	LDA 4
The Accumulator (A)	5

Address	00	01	02	03	04
Instruction / Data	LDA 4	SUB 5	STA 6		5
Address	05	06	07	08	09
Instruction / Data	2				

Because the first cycle has now been completed, the program counter is updated to 1, ready for the next cycle.

The Program Counter (C)	1
The Memory Address Register (MAR)	0
The Memory Data Register (MDR)	LDA 4
The Instruction Register (IR)	LDA 4
The Accumulator (A)	5

Address	00	01	02	03	04
Instruction / Data	LDA 4	SUB 5	STA 6		5
Address	05	06	07	08	09
Instruction / Data	2				

FETCH STAGE

Now the second cycle begins and the next instruction is fetched from memory.

The program counter is now at 1 and the value of 5 is in the accumulator.

Computer Science UK Membership Site Licence: Do not share outside your centre.

The Program Counter (C)	1
The Memory Address Register (MAR)	
The Memory Data Register (MDR)	
The Instruction Register (IR)	
The Accumulator (A)	5

Address	00	01	02	03	04
Instruction / Data	LDA 4	SUB 5	STA 6		5
Address	05	06	07	08	09
Instruction / Data	2				

Like before, the counter value is copied to the MAR.
This represents the address of the next instruction to be fetched.

The Program Counter (C)	1
The Memory Address Register (MAR)	1
The Memory Data Register (MDR)	
The Instruction Register (IR)	
The Accumulator (A)	5

Address	00	01	02	03	04
Instruction / Data	LDA 4	SUB 5	STA 6		5
Address	05	06	07	08	09
Instruction / Data	2				

The program uses the value in the MAR to look up the appropriate instruction by looking in address 01.

The Program Counter (C)	1
The Memory Address Register (MAR)	1
The Memory Data Register (MDR)	
The Instruction Register (IR)	
The Accumulator (A)	5

Address	00	01	02	03	04
Instruction / Data	LDA 4	SUB 5	STA 6		5
Address	05	06	07	08	09
Instruction / Data	2				

This instruction is then loaded into the MDR...

The Program Counter (C)	1
The Memory Address Register (MAR)	1
The Memory Data Register (MDR)	SUB 5
The Instruction Register (IR)	SUB 5
The Accumulator (A)	5

Address	00	01	02	03	04
Instruction / Data	LDA 4	SUB 5	STA 6		5
Address	05	06	07	08	09
Instruction / Data	2				

...and then moved to the IR, ready to be decoded by the CPU.

DECODE STAGE

Now that the CPU has fetched the next instruction, it needs to decode it.

The Program Counter (C)	1
The Memory Address Register (MAR)	1
The Memory Data Register (MDR)	SUB 5
The Instruction Register (IR)	SUB 5
The Accumulator (A)	5

It does this by looking up the IR instruction in its character set.
It recognises SUB 5 as meaning SUBTRACT the contents of address 05 from the accumulator.

The Instruction Set

Instruction	Purpose
ADD	Add numbers together
SUB	Subtract numbers from each other
STA	Store a value
LDA	Load a value
BRA	Branch to a different part of the program (often used to repeat code)
OUT	Output a value (after processing)

The CPU has now decoded the first instruction and is ready to execute it!

EXECUTE STAGE

Computer Science UK Membership Site Licence: Do not share outside your centre.

Now that the CPU has made sense of the instruction, it now needs to execute the instruction.

The instruction is execute and the value of address 05 is SUBTRACTED from the value in the accumulator.

The Program Counter (C)	1
The Memory Address Register (MAR)	1
The Memory Data Register (MDR)	SUB 5
The Instruction Register (IR)	SUB 5
The Accumulator (A)	5

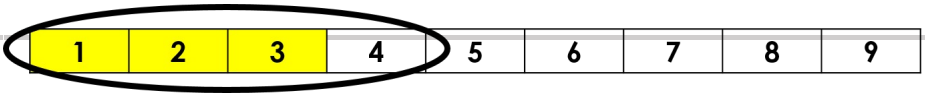
Address	00	01	02	03	04
Instruction / Data	LDA 4	SUB 5	STA 6		5
Address	05	06	07	08	09
Instruction / Data	2				

The Program Counter (C)	1
The Memory Address Register (MAR)	1
The Memory Data Register (MDR)	SUB 5
The Instruction Register (IR)	SUB 5
The Accumulator (A)	3

The program has now been run and the result is **3**.

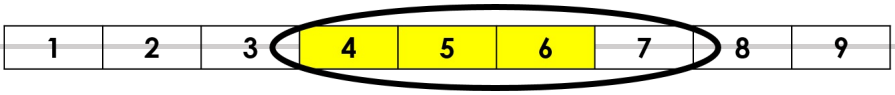
Questions *(The question zone you choose must either match your target grade or be higher!)*

Question Zone 1-3



1. State the assembly language instruction used to stores data in the accumulator. [1]
2. Define the term 'instruction set'. [2]
3. Describe the role of each of the 5 main CPU registers. [5]

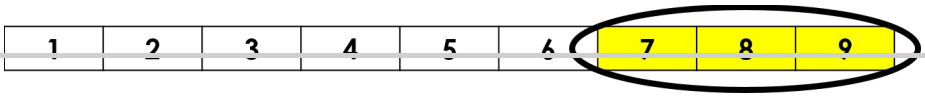
Question Zone 4-6



1. Describe the role of each of the 5 main CPU registers. [5]
2. Using the example program in the table provided, describe the purpose of the program making reference to the instructions and data given. [4]
3. Explain the difference between the MAR and the MDR. [2]

Address	00	01	02	03	04
Instruction / Data	LDA 4	ADD 5	STA 6		5
Address	05	06	07	08	09
Instruction / Data	9				

Question Zone 7-9



1. Explain the difference between the MAR and the MDR. [2]
2. Using the example program in the table above, explain the process involved in fetching, decoding and executing the first instruction. Make sure that you reference the major registers in your answer. [5]
3. Discuss the importance of Von Neumann in the development of computer technology. [5]

Computer Science UK Membership Site Licence: Do not share outside your centre.

Checklist:

- Date and title, clearly presented
- Spelling & grammar checked
- Question numbers in the margin
- Handwriting neat & legible
- Punctuation / Capital letters

Keywords / Key Terms:

Register: *A small amount of fast storage in the CPU.*

CPU Architecture: *The design of the CPU (how it has been constructed).*

Accumulator: *A specialised CPU registered which stored the intermediate results of a F-D-E cycle.*

Computer Science UK Membership Site Licence: Do not share outside your centre.

Stick answer sheet here

State/Identify/Give/Name: Simply label a diagram, fill out a table or write a few words
Describe: Describing is 'saying what you see' (E.G.: A computer will have a CPU, Primary and Secondary storage etc)
Explain: Explaining is 'saying WHY/HOW something is like that'. (E.G.: A computer will have a CPU so that it can process all of the data the computer needs to perform a range of tasks. Primary and Secondary storage is needed because...)
Discuss: Discussing is 'looking at two sides of an issue, weighing up the two views and giving a conclusion'. Often these require a mini essay answer. (E.G.: New technology could be seen as being bad for the environment because..., but on the other hand, new technology has led to... In conclusion I believe that...)
Describe/Explain/Discuss **using examples**: Finally, if you are asked to give examples in any of these types of questions – YOU MUST **GIVE EXAMPLES**!

Computer Science UK Membership Site Licence: Do not share outside your centre.

Reflections **Score:** / **Percentage:** % **Grade:** **Progress** On / Above / Below

What Went

- I demonstrated a good level of understanding.
- I responded to the command words effectively.
- My answers were detailed / were written in depth.
- My work was well presented / legible.
- My answers effectively incorporated technical terminology.
- My responses were well structured / organised.
- My revision strategy was effective as I showed depth of understanding in my answers.
- My answers contained enough points / examples / explanations to achieve the marks available.

Even Better

- My answers need to be more accurate.
- I must respond correctly to the command words.
- My answers need more detail / greater depth.
- I must take greater care over my work / write neatly.
- I must incorporate key terminology into my answers.
- I must better organise my answers to improve its clarity.
- I need to improve my revision strategy as I did not demonstrate a depth of understanding in my answers.
- My answers didn't contain enough points / examples / explanations to achieve the marks available.

Further thoughts: